

Efficient Semifield Convolutions

Peter Adema
14460165

Honours Thesis Extension
Credits: 6 EC

Bachelor *Kunstmatige Intelligentie*



University of Amsterdam
Faculty of Science
Science Park 900
1098 XH Amsterdam

Supervisor
Dr. ir. R. van den Boomgaard

Informatics Institute
Faculty of Science
University of Amsterdam
Science Park 900
1098 XH Amsterdam

Semester 2, 2024-2025

Abstract

The convolutional operator is often used within computer vision applications, as well as in Convolutional Neural Networks from machine learning. While the linear convolution is well-optimised, there are many operators with a strong structural resemblance (such as morphological dilation) that have no implementation for use in machine learning or a computationally inefficient one.

Using the mathematical concept of semifields, many operators similar to linear convolution can be defined as convolutions in a different semifield, referred to as 'semifield convolutions'. By leveraging Just-In-Time (JIT) compilation with Numba-CUDA and PyTorch, we can then JIT-compile efficient CUDA operators for many semifields and provide an interface for defining custom semifields in which various convolutions can be performed.

The result of this report is the package [pytorch-nd-semiconv](#), which can match the performance of built-in PyTorch operators in some cases (when the reduction operator is like *max*) but not in others (when the reduction operator is like *+*).

This report is an extension of research done into anisotropic dilations, with code for this and the main report being available in a [GitHub repository](#).

Acknowledgements

I would like to thank my supervisor Dr. ir. R. van den Boomgaard for his time in reviewing this report, as well as his contributions to my main report and thesis. I would also like to thank my friends and family for their support during this period.

Contents

1	Introduction	3
2	Backpropagation for semifield convolutions	4
2.1	PyTorch reduction operators	4
2.2	Efficient derivatives for custom kernels	6
2.3	Selection-based semifield addition	7
2.4	Invertible semifield addition	7
3	CUDA semifield convolutions	8
3.1	Compute Unified Device Architecture	8
3.2	Parallel Thread Execution and SASS	9
3.3	Numba-CUDA Just-In-Time compilation	9
3.4	PyTorch Integration	10
3.5	Dispatch overhead	10
3.6	Atomics	11
3.7	Shared Memory	11
4	Results	12
4.1	Verification	13
4.2	Timing setup selection-based semifields	13
4.3	Performance of invertible semifields	14
4.4	Performance of selection-based semifields	14
5	Conclusions	16
5.1	Discussion	16
5.2	Contributions	17
5.3	Further research	17
5.4	Reproducibility	17
	Appendix	19
A	Performance results on other GPUs	19

Chapter 1

Introduction

Within computer vision, one of the most often used operators for image analysis is the convolutional operator [1]. This operator functions by sliding a window over the input image and taking a weighted average using a fixed set of weights (which are often referred to as the convolutional kernel). As a formula, we can consider the convolution $*$ to be a reduction in a region $\mathcal{Y}$ around a point x :

$$(F * G)[x] = \sum_{y \in \mathcal{Y}} F[x - y] G[y] \quad (1.1)$$

However, besides the standard (linear) convolution, there are operators from computer vision and other fields that share a similar structure. One of these is the common morphological dilation [2], effectively a weighted maximum:

$$(F \boxplus G)[x] = \max_{y \in \mathcal{Y}} (F[x - y] + G[y]) \quad (1.2)$$

In terms of implementation, both of these operators share many similarities. In effect, they differ only in the specifics of the operator which combines the image and the kernel, and in the operator which performs the reduction. Considering these similarities, we can choose to define a version where these are left generic:

$$(F \circledast_S G)[x] = \bigoplus_{y \in \mathcal{Y}} F[x - y] \otimes G[y] \quad (1.3)$$

Such a generic definition corresponds best with the mathematical concept of a semifield or semiring [3], whereby the operators $\oplus$ and $\otimes$ are defined to behave similarly to scalar $+$ and $\times$. Eq. 1.3 allows us to use the same mental framework to cover a wide variety of operators, e.g. by using the tropical-max [4] or ' T_+ ' semifield/semiring for Eq. 1.2, or a simple linear field for Eq. 1.1. Other examples (e.g. root semifields) are mentioned in the main report [5].

However, while defining a convolution-like operator using semifields allows for great mathematical generality, there are currently no software packages for applying such 'semifield convolutions'. Furthermore, while the standard linear convolutions are well-optimised in packages such as CUDNN, even fairly common variations such as the dilation from Eq. 1.2 have received far less attention. As a result, even in the cases where an implementation exists within the context of machine learning (such as for dilation, from Morphpool [6]), the efficiency and speed of these implementations may be suboptimal.

This report is an extension to the main report on a specific application of dilations [5]. Considering the number and scale of experiments required to be run for that report, it was decided that a more efficient package for performing semifield convolutions would greatly assist that and future research. As such, the package [pytorch-nd-semiconv](#) was written to fill this gap, with this report serving as documentation and explanation for some of the choices made.

Chapter 2

Backpropagation for semifield convolutions

To perform semifield convolutions within the context of deep learning, we must ensure that we can take the gradient with respect to our inputs in an efficient manner for all operations we seek to perform. The primary challenge in this case is not necessarily the gradient for $\otimes$, as this is a scalar function applied once within each path in the computational graph. Instead, the challenge lies in calculating the derivative of the semifield summation $\oplus$, as all terms within this reduction could influence the gradient of all others. While it would be possible to simply perform backpropagation through every instance of $\oplus$ separately, this would be computationally expensive and likely numerically unstable. As such, we would like to define semifield operators for cases where a more efficient method for backpropagation through $\oplus$ can be found.

We have identified three cases in which this is possible and will discuss them in order of increasing implementation complexity. First, we will examine the possibility of using operators provided by PyTorch to implement semifield convolutions. Afterwards, we will show the two common cases for which a reduction has a simple analytical derivative, namely for those operators $\oplus$ which act like *max* and those which are invertible.

2.1 PyTorch reduction operators

The most straightforward method for implementing semifield convolutions for PyTorch models is to use the reduction operators provided by PyTorch itself. These operators have their derivatives given by PyTorch and are integrated into the PyTorch automatic differentiation system such that writing a backward pass is not required.

The challenge when using PyTorch operators lies not in their derivatives but in their correct application, as we must first construct a version of the input containing the sliding windows we wish to reduce over.

A built-in method for accomplishing this is `nn.Unfold`, an operator which examines sliding windows from the input and copies each window to a new buffer, allowing subsequent operators to work with the window axes of this new buffer. This copying is rather suboptimal; if we take a relatively small 32×32 image and extract relatively small 5×5 windows from it, then the result is more than 19 times the size of the image. Furthermore, the process of copying is itself somewhat expensive due to the scale of the result.

As such, we would like to perform a similar operation — transforming the input image such that we have access to sliding window axes to reduce over — but without requiring a copy. This transformation is possible by creating an adjusted view of the input image: some other libraries include this feature by default, but it is not exceedingly complex to replicate.

The main idea of using such a view is to adjust the strides (step sizes within the underlying data array) such that, when moving to the next window, the view takes a step equal to the convolutional STRIDE instead of the full size of the window. In Fig. 2.1, we can see how a view into a 1D-array with STRIDE = 1 and size-4 kernels causes subsequent windows to overlap:

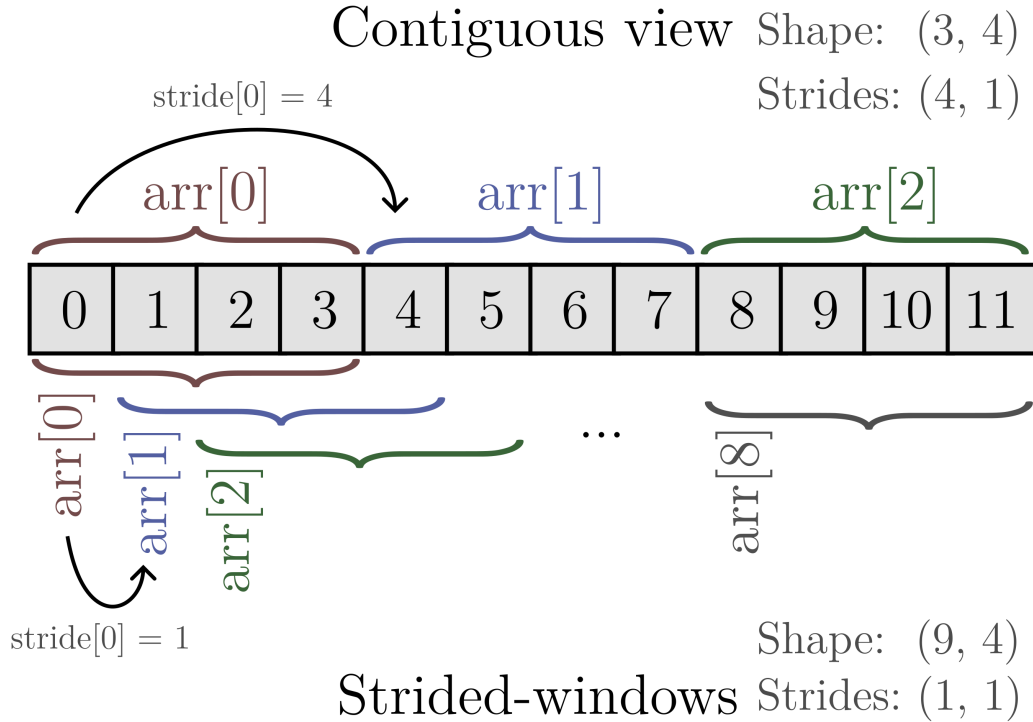


Figure 2.1: Two views onto the same underlying data: at the top, a contiguous view (the typical view for PyTorch arrays), and at the bottom, a strided-windows view (allowing for extracting windows without copying).

By applying a broadcasting version of $\otimes$ to the strided view and the convolutional kernels, we can obtain the intermediaries to be reduced. Subsequently, we can apply the reduction operator along the window axes (axis 1 in Fig. 2.1) to efficiently calculate $\oplus$ for all cases where $\oplus$ and $\otimes$ can be expressed as one or more PyTorch operators.

However, this approach still has a major downside in terms of memory usage and cache locality. There is currently no method for fusing the $\otimes$ and $\oplus$ operations in PyTorch, and the intermediary results of $\otimes$ must therefore be manifested in their entirety. As a result, this method still suffers from greatly increased memory use and also accesses memory inefficiently. Since all results of $\otimes$ are stored before $\oplus$ begins, the initial results of $\otimes$ are usually flushed from the cache by the beginning of $\oplus$, causing cache-misses and further slowdown.

To mitigate this, we must devise custom, fused kernels that combine $\otimes$ and $\oplus$ into a single operation. Such kernels, however, require manual calculation of the derivative rules. As such, the following two sections will discuss two methods for deriving the reduction $\oplus$: when $\oplus$ is a selection, or is invertible.

2.2 Efficient derivatives for custom kernels

If we consider the operations that take place within the convolutional operation, then the computational graph for any one point in the result is an unbalanced binary tree, with a sequence of $\oplus$ operations along the spine constituting the reduction. If we would truly wish to derive this computational graph as-is, then the gradients propagated to the bottom of the tree may suffer from numerical instability due to the overly high number of operations to backpropagate through. Instead, we would prefer cases in which we could skip deriving through the reduction and calculate the derivative of the result with respect to any term in the reduction directly. This difference is illustrated in Fig. 2.2, where the right computational graph shows these direct derivatives:

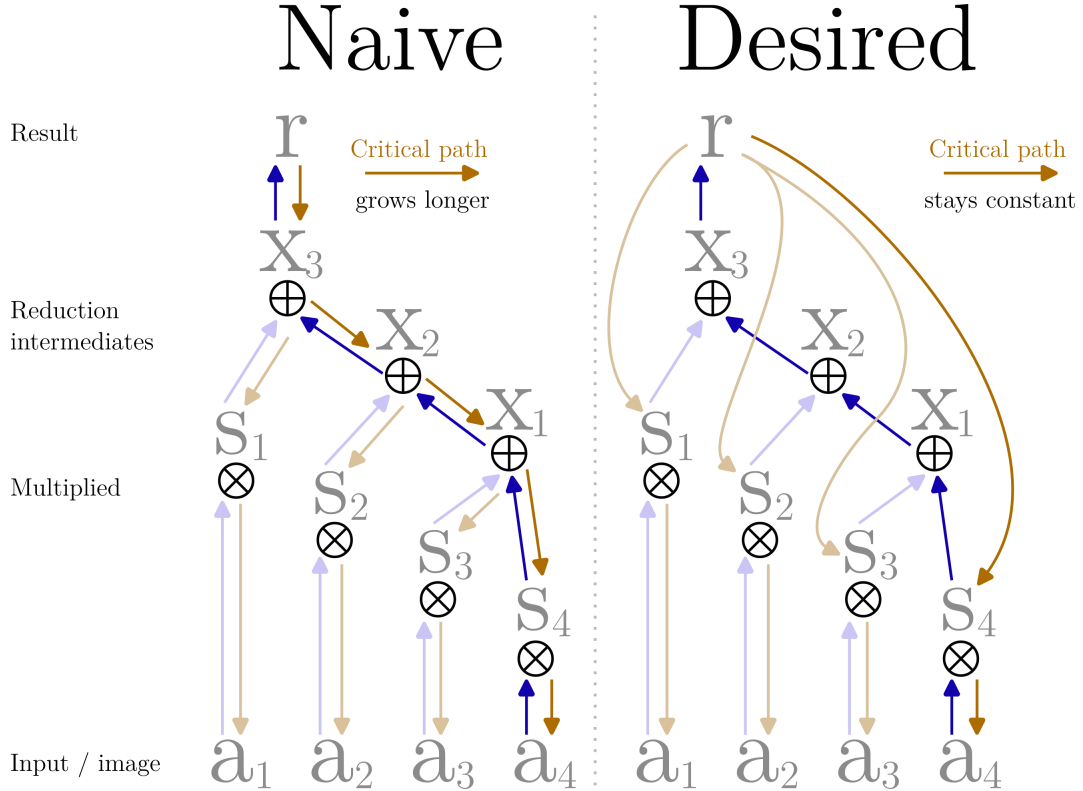


Figure 2.2: Two examples of computational graphs for a semifield convolution at a single point, with the left graph showing a naive implementation of the backwards graph and the right graph showing a more efficient, desired version. The right graph is only relevant if $\frac{\delta r}{\delta s_n}$ is easily computable for any n .

Unfortunately, $\frac{\delta r}{\delta s_n}$ is not easily computable for any n in all semifields. In order to attain the desired behaviour for the backward pass, we must therefore place additional constraints on the operator $\oplus$; it must either:

- Act as a selection — similar to max and min — where r is always takes on the value of one of the inputs s . See Sec. 2.3 for details.
- Be invertible (as well as commutative and associative), such that when $r = x \oplus s$, then x can be found given r and s . See Sec. 2.4 for details.

These two cases are the only cases for which a custom kernel is implemented in the package written alongside this report, corresponding with `SelectSemifield` and `SubtractSemifield` respectively.

2.3 Selection-based semifield addition

When $\oplus$ acts as a selection operator, its result is always equal to one of its two operands. The typical examples of selection operators are max and min, which always select the higher and lower operands, respectively. However, it is conceivable that one may wish to perform a maximum-like operation using an alternative ordering, hence this generalisation.

The advantage of selection-based semifield addition is that the result of the reduction $\bigoplus$ must also be equal to one of its inputs (an s from Fig. 2.2) while having no relation to any of the other inputs. As such, we can see that the derivative $\frac{\delta r}{\delta s_n}$ is equal to 1 for the s_c which r takes its value from, and 0 for all s_i where $i \neq c$. As such, letting the backpropagation 'skip over' the reduction would only require knowing the value of c (the index of the selected s_c).

While it is possible to recompute this c using a second forward pass during backpropagation, we can — using a small amount of auxiliary memory — memoise this index c during the initial forward pass. By storing this c (also known as the provenance index), the backward pass only needs to update the gradient of the value at index c and can ignore all other terms in the reduction. The backward pass, therefore, only requires recomputing s_c , not the other s 's.

2.4 Invertible semifield addition

For all other operators that we would wish to use as $\oplus$, simple memoisation of the result will not suffice. There is, however, still a class of operators which leads to relatively simple backward graphs. If we examine again the naive backwards graph from Fig. 2.2, we can see that, although the path of the gradient is relatively long for most s_i , the path for s_1 is relatively short. The last value added in the reduction only requires the derivative of one instance of $\oplus$, greatly simplifying the calculation. However, for any specific ordering of the values $s_1 \dots s_n$, there is only one instance of this 'easy case'. To solve this, we could require an associative and commutative $\oplus$, and write for any s_i [7]:

$$r = s_1 \oplus s_2 \oplus \dots s_{i-1} \oplus \mathbf{s}_i \oplus s_{i+1} \oplus \dots s_{n-1} \oplus s_n \quad (2.1)$$

$$= (s_1 \oplus s_2 \oplus \dots s_{i-1}) \oplus \mathbf{s}_i \oplus (s_{i+1} \oplus \dots s_{n-1} \oplus s_n) \quad (2.2)$$

$$= \mathbf{s}_i \oplus (s_1 \oplus s_2 \oplus \dots s_{i-1} \oplus s_{i+1} \oplus \dots s_{n-1} \oplus s_n) \quad (2.3)$$

$$= \mathbf{s}_i \oplus x_{\setminus i} \quad (2.4)$$

In other words, we can rewrite the reduction for any s_i into this 'easy case', where the result is equal to s_i combined with the accumulation of all other values. The challenge then becomes finding this $x_{\setminus i}$: simply performing the reduction for every i separately would be highly inefficient, so a more efficient solution is required. If $\oplus$ is invertible, then the solution is straightforward: by Eq. 2.4 we know that $r = s_i \oplus x_{\setminus i}$, so if we recalculate s_i and memoise r (from the forward pass), then $x_{\setminus i}$ can be found using the inverse of $\oplus$: $x_{\setminus i} = r \oplus^{inv} s_i$. Given both s_i and $x_{\setminus i}$, we can then easily calculate the gradient of Eq. 2.4.

In practice, the restriction of invertibility is not overly constraining. All fields are necessarily invertible (we can calculate $x_{\setminus i}$ by taking the element which is the additive inverse of s_i and seeing that $x_{\setminus i} = r \oplus s_i^{inv}$). Furthermore, even many semifields which are not themselves fields have inverses, such as the log semifields $L_{\pm\mu}$ and the root semifields R_p . In these cases, by replacing the $+$ within the semifield addition with $-$, we can obtain the inverse.

Chapter 3

CUDA semifield convolutions

Armed with an understanding of the types of semifield convolutions where a gradient can be calculated in a reasonably efficient manner, we now turn to the task of efficiently implementing these operations as programs that can run on a modern NVIDIA GPU. First, we will discuss how programs that can run on an NVIDIA GPU are represented, as well as how such programs can be written in Python using Numba-CUDA. Subsequently, we will discuss how the overhead associated with dispatching GPU programs (GPU 'kernels') can be reduced and how some features of CUDA can be leveraged to improve performance.

3.1 Compute Unified Device Architecture

Commonly referred to as CUDA, this programming model allows for describing operations in an inherently parallel fashion [8]. Typically, CUDA programming involves splitting the parallelisable task into its smallest parts and spawning a thread for every individual task (for machine learning, often one thread per position in the target array). Threads are then hierarchically grouped, with 32 threads forming a 'warp' and a variable¹ number of warps forming a 'block'².

Within a warp, threads can communicate using special instructions that transfer data between registers of a warp. For more structured cooperation, threads within a block can access 'shared memory', which is most similar to a fast version of heap-allocated memory for CPU threads [8]. Blocks can be further arranged into 'clusters' if more warps are required to work on the problem than can fit within one block, but this is rarely used.

In terms of actual execution, an operation (such as a semifield convolution) consists of blocks arranged into a 'grid'. When a grid is executed, some of its blocks are assigned to a Streaming Multiprocessor (SM), where all the block's threads are run from start to finish. In an SM, warps are issued instructions, with all threads in a warp executing (or skipping) the same instructions. In a sense, warp instructions are highly optimised and flexible versions of predicated SIMD instructions³. However, besides using this data parallelism to accelerate computation, CUDA devices can also leverage concurrency (akin to CPU hyperthreading) to hide memory latencies. Every computational unit ('scheduler', e.g. 4/SM) can house several (up to e.g. 16) warps and can retain optimal performance so long as one of the warps is always eligible for execution.

¹On most hardware, a block may have between 1 and 32 warps [9] (32 to 1024 threads).

²Also called a 'thread block', or a 'cooperative thread array' in PTX documentation [10].

³The model of warps is also referred to as Single Instruction Multiple Threads (SIMT)

3.2 Parallel Thread Execution and SASS

Besides describing a device architecture, the term 'CUDA' is often used to refer to the CUDA C extension to the C programming language, which allows users to write programs that run on CUDA devices in a dialect of C or C++. However, just like C does not run directly on a CPU, CUDA C itself cannot run on CUDA devices: it must first be compiled to machine instructions.

The assembly format of choice for CUDA devices is Streaming Assembler (SASS), which compiles into executable bytecode. Most compiler frontends that target CUDA devices do not output SASS directly, however. Instead, a more cross-compatible assembly format, named Parallel Thread Execution (PTX), is used, and an NVIDIA transpiler (`ptxas`) is used to convert PTX into SASS at runtime [10]. The idea behind this construction is similar to that of VM-based languages, where developers can write programs in a high-level language (such as CUDA C), compile to a backwards- and forwards-compatible format for distribution and have their code run on a wide range of devices.

3.3 Numba-CUDA Just-In-Time compilation

CUDA C is not the only frontend for PTX; the Numba-CUDA package [11] provides an alternative based on Python. By providing the same intrinsics that CUDA C offers, Numba-CUDA enables writing programs for CUDA devices in relatively standard Python. Unlike CUDA C, applications using Numba-CUDA are not distributed with their GPU code compiled to PTX but instead leave this compilation step as a runtime task as well (in addition to the task of transpiling the PTX to SASS and assembling the SASS).

Just-In-Time (JIT) compilation refers to the above technique of delaying compilation to be a one-off runtime cost instead of being a preparatory step. JIT compilers and applications have the advantage of being able to specialise at runtime — certain aspects of the input can be assumed to be constant (such as the shape of an image or the dilation of a convolution), providing more room for optimising compilers to improve performance. The primary downside of JIT compilers is usually taken to be the increased start-up times associated with runtime compilation. However, these costs are negligible in machine learning contexts, where training can consist of thousands or millions of iterations.

Due to the reduced complexity of programming in Python, as well as the flexibility of runtime specialisation, it was chosen to write the packages for this report using Numba-CUDA as the PTX frontend instead of CUDA C.

The full compilation pipeline is visualised for clarity in Fig. 3.1:

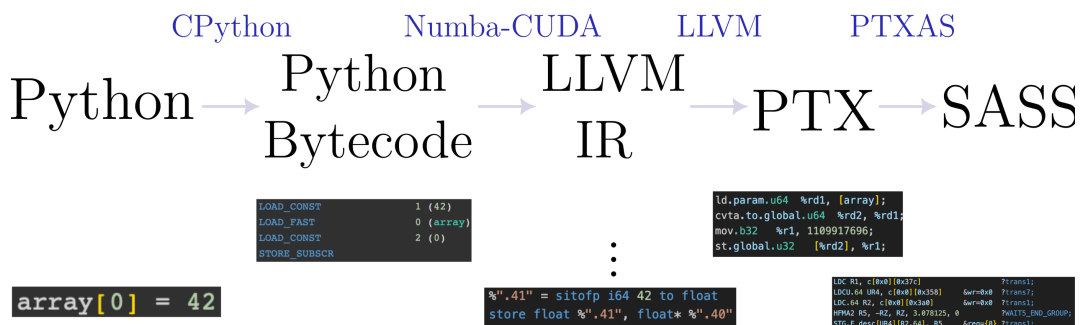


Figure 3.1: Compilation pipeline from Python source code to SASS bytecode.

3.4 PyTorch Integration

Many projects performing research in machine learning (including this one) use the PyTorch framework to arrange operations into a computational graph for backpropagation or compilation. However, while a sequence of PyTorch built-in operators can typically be traced to form such a graph, this is not the case for custom CUDA operators. In these cases, more care is required in order to use these operators within the PyTorch automatic differentiation system.

Using `torch.library`, custom operators can be defined such that PyTorch tracing systems (`dynamo`) can properly integrate the operators into their graphs. However, writing such code in Python can lead to a significant per-call dispatch overhead for these operators, as various conversions and checks are required during the invocation of a custom Python operator, making them slower than their built-in counterparts. Furthermore, while Numba-CUDA supports the CUDA Array Interface and can interact with PyTorch data, using this Python interface also causes slowdowns. The total additional overhead can cause operators to be significantly slower (five times slower for this report, as shown in Fig. 4.1). As such, resolving this slowdown is also an important factor to consider before attempting to use these operators to train models.

3.5 Dispatch overhead

One approach to resolving this slowdown is to rewrite the operators as compiled (C++) extensions to the PyTorch framework. The primary downside of this approach would be that it would require source code for the operators to be available for compilation, and the generic nature of semifield convolutions makes them very challenging to prepare for ahead-of-time compilation. This can be resolved by making the C++ extensions also JIT-compiled. By using Numba-CUDA to compile dynamic Python code, the resulting PTX can be embedded in a C++ function without requiring much knowledge of its workings. This construct, where Numba-CUDA is used to JIT-compile PyTorch extensions, is implemented by the `pytorch-numba-extension-jit` package written for this report and can eliminate much of the Python-induced overhead from custom operators in PyTorch computation graphs.

However, a more effective method is available for training models. In general, the nature of any overhead within the training process originates from the CPU being unable to start GPU programs faster than the GPU can complete them. While this cannot be avoided in cases where the programs to be executed differ between runs, training models in machine learning typically involves running a fairly static forward and backward pass for many iterations. In such cases, an alternative approach named CUDAGraphs can allow a CUDA GPU to record a series of operations and then repeat them with only a single (more expensive) invocation (like a 'macro' from productivity software). Using CUDAGraphs, dispatch overhead from custom operators can be eliminated entirely, without even resorting to JIT-compilation of C++ extensions.

Since the main report is primarily concerned with training and evaluating various machine learning models, CUDAGraphs is used without additional C++ compilation to reduce dispatch overhead. For other cases, where the process of CUDAGraphs tracing is too lengthy or the additional invocation cost is too high, C++ extension compilation may still be an effective method.

3.6 Atomics

Within a CUDA GPU, numerous features are designed to accelerate various computational tasks. Most of these features could not be effectively applied for semifield convolutions, except for the feature of atomic additions. While the forward pass of both a selection-based semifield and an invertible semifield can be written with standard Python constructs, the backward pass of both requires aggregating the gradients for various positions in the image and kernel. It is possible to do this aggregation using a convolution-like operation in reverse, i.e. for every position in the input, sum over the gradients with respect to every output position that could have accessed it. However, this is suboptimal in the case of selection-based semifields, as every output position only corresponds with at most one input position. Atomic additions allow threads to cooperate on a summation: multiple threads can concurrently perform an addition on a memory address (e.g. a part of the gradient for the input), and the hardware will guarantee that all summations are performed in some consistent order. This is important, as several threads attempting to add values to a specific address might otherwise overwrite each other’s work. By leveraging atomics, the backward pass for selection-based semifields can be performed efficiently, in some cases even as fast as the built-in PyTorch max-pooling (see Fig. 4.1)

3.7 Shared Memory

Besides atomic operations, another feature often used in CUDA programs is ‘shared memory’, which allows threads in a block to access a shared region of memory to cooperate on a task. Often, shared memory is used to store intermediate results or to cache input values for more efficient access [8].

For semifield convolutions, however, shared memory is challenging to apply effectively. The first use case — caching intermediates — does not play a significant role within semifield convolutions, as there are no clear intermediate steps in the calculation. Breaking up the window and assigning multiple threads per output position is an option, but it resulted in decreased performance during testing. As such, the only possible use case for shared memory would be in caching inputs.

The argument in favour of this use case is that accessing uncached, global memory is slow if all memory accesses are not clustered together (in 128 bytes), whereas a looser set of limitations applies for shared memory accesses [8]. As such, storing the input image or kernel might allow for more efficient access to it. While this seems reasonable, it is not the case for convolutions with a group size of 1. It is possible to write the convolution in a manner where memory access is mostly linear and predictable, after which the automatic caching system (akin to CPU L1/L2 cache) can correctly keep all relevant data in the cache. As such, while many variations on shared memory caching were tried — from copying the entire relevant parts of the image/kernel to maintaining a local sliding window of the image — none could outperform the automatic cache, and most performed significantly worse. However, many of the performance tests focused on optimising poolings for the main report, which have an input group size of 1; with larger groups, the results may differ.

Considering the above, it was chosen to not use any shared memory or other synchronisation tools within the implementations of the semifield convolutions, using only atomic addition as a non-standard (compiler intrinsic) feature.

Chapter 4

Results

Created using the derivative schemas from Chapter 2 and the technical tools described in Chapter 3, the package `pytorch-nd-semiconv` implements the three types of semifield convolutions discussed and provides integrations for use with the PyTorch machine learning framework. The operators work with arbitrary dimensionality (e.g., 1D signals, 2D images, or 3D imaging data). Using this package, verification experiments were conducted and timings were recorded to examine the correctness and performance of the resulting code. As a usage example, consider the following dilation-based pooling over 5 channels:

```
1 import pytorch_nd_semiconv as semiconv
2
3 dilation = semiconv.GenericConv(
4     semiconv.QuadraticKernelSpectral2D(
5         in_channels=5, out_channels=5, kernel_size=3
6     ),
7     semiconv.SelectSemifield.tropical_max().lazy_fixed(),
8     padding="same",
9     stride=2,
10    groups=5,
11 )
```

The package is fairly modular in that both the kernel and the convolutional operator can be freely specified within the `GenericConv` wrapper, with some example implementations for 2D kernels being included. Additionally, we can see that the T_+ semifield has a standard definition present in the package. This is the case for all semifields discussed in the main report, but constructing new semifields for defining convolutional operators is also possible. The equivalent code for constructing a T_+ semifield would be the following:

```
1 my_tropical_max = semiconv.SelectSemifield(
2     add_select=lambda left, right: left < right,
3     times=lambda img_val, kernel_val: img_val + kernel_val,
4     d_times_d_img=lambda _i, _k: 1.0,
5     d_times_d_kernel=lambda _i, _k: 1.0,
6     zero=-math.inf,
7 ) # equivalent to semiconv.SelectSemifield.tropical_max()
```

Here, we provide a predicate for $\oplus$ that determines when to select the rightmost argument over the left, a definition and its partial derivatives for $\otimes$ and the semifield zero to construct a selection-based semifield. The result can then be used to compile a custom CUDA operator for use in e.g. `GenericConv`.

For more examples and usage guidelines, see the [documentation](#).

4.1 Verification

To ensure that the results for the main report accurately represent the characteristics of semifield convolutions, it was essential to verify that the operators in `pytorch-nd-semiconv` are correct and abide by expectations.

Verification of these operators was done in three parts. First, a set of simple experiments was done using fixed data, with results and gradients verified to be identical to their manually calculated counterparts. Secondly, theoretical characteristics of morphology were used to verify the correctness of dilations (T_+) and erosions (negated T_-) specifically. Finally, a variety of random inputs were used to verify that the CUDA implementations behaved identically to their PyTorch broadcasting counterparts.

Regarding the fixed data experiments, interested readers may consult the Jupyter Notebook `groups_test.ipynb` to view the results of the operators on some simple data and using convolutional groups.

For the morphological characteristics, it was chosen to focus on verifying the extensivity of closings (i.e., that the results of a closing are always $\geq$ the input) as well as the dual anti-extensivity of openings ($\dots \leq$). Using both random and fixed data, these characteristics were verified to hold in `plot_adjunction.ipynb`.

Finally, an extensive test covering 100 random configurations for input dimensionality, input and kernel size, stride, dilation, and padding was run, where the results and gradients of the PyTorch broadcasting implementation were verified to be identical to those of the corresponding implementation using CUDA code. `ops_equivalent.py` showed that the implementations were equivalent in nearly all¹ cases, strengthening the case for the correctness of the operators.

4.2 Timing setup selection-based semifields

After performing some tests to verify the correctness of the implementations in `pytorch-nd-semiconv`, additional experiments were run to determine the speed of various versions of the same semifield convolution. A set of trials was run using T_+ as the representative for selection-based semifield convolutions using the following four configurations (without using CUDAGraphs):

1. 32×32 input, 3×3 kernel with stride 1
2. 32×32 input, 3×3 kernel with stride **3**
3. 32×32 input, **7×7** kernel with stride 3
4. **256×256** input, 7×7 kernel with stride 3

For other parameters, the defaults are `PADDING = 0` and `DILATION = 1`. While experiments were initially run using the same NVIDIA RTX 5090 GPU as the main report, Appendix A shows similar results using other GPUs.

¹The derivative of semifield multiplication does not always consider the absorbing nature of the semifield zero: $\frac{\delta(x+\infty)}{\delta x} = 1$ in PyTorch, even when changing x would not change the result of $x+\infty$. As such, when the kernel contains a semifield zero (due to it being set as such) or the input does (due to e.g. padding), the gradient can differ between implementations. For example, if the entirety of the convolutional receptive field is within padded area, then the custom CUDA operator implementation of a T_+ convolution will correctly recognise that the kernel has no influence on the output, while the PyTorch broadcasting operators will incorrectly assign a nonzero gradient to one of the elements of the kernel (typically the first).

4.3 Performance of invertible semifields

While invertible semifields also have convolutional operators implemented in the package (under `SubtractSemifield`), significantly less time was devoted to performance analysis and optimisation for these operators, as they did not play a major role in the main report. As a consequence, the actual performance of e.g. the linear convolution as implemented by `pytorch-nd-semiconv` is rather poor, ranging from 3x slower for small inputs to orders of magnitude slower for large inputs. The primary difference in performance comes from the implementation of the backward pass; while using atomics is a good strategy for selection-based semifields — where only one input can receive a gradient — a different strategy is required for invertible semifields (e.g. a ‘reverse convolution’ as described in Sec. 3.6). However, other strategies were not examined due to a lack of time and it not being necessary for the main report.

In short, the performance of invertible semifields is poor and was therefore not extensively measured; more performance work is required in this area.

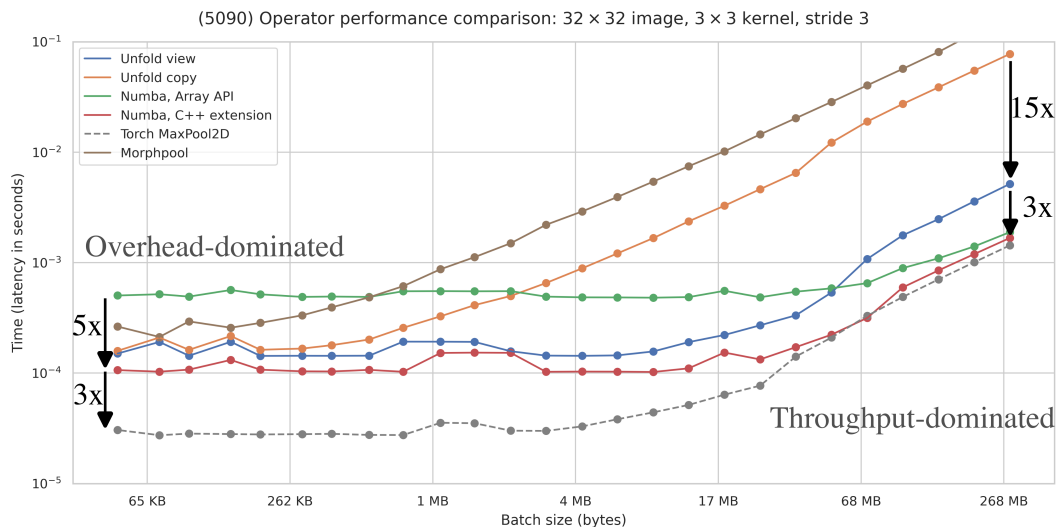
4.4 Performance of selection-based semifields

On the following page, Fig. 4.1 shows the performance of various versions of a T_+ convolution (equivalent to a morphological dilation) on the four setups from Sec. 4.2, with the second configuration shown in large. The left column shows the average latency of the operator in seconds, while the right column shows the peak memory usage of the operator (including the memory required to store the result). An additional implementation named ‘Morphpool’ is also shown, which originates from the correspondingly titled paper by R. Groenendijk [6].

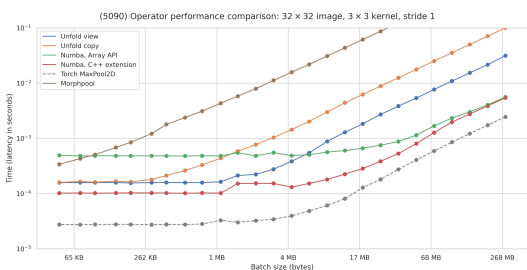
We can see that the results are mostly in line with expectations, with performance for small batch sizes determined mainly by the level of overhead incurred by the operator, and performance for large batch sizes determined by the maximum throughput of the operator. We can see that the implementation using `nn.Unfold` (Unfold copy) performs poorly, while the C++ extension performs the best, even comparably with a simple max-pooling. Additionally, while the implementation from ‘Morphpool’ does not use a large amount of auxiliary memory, it is also the slowest implementation (slower than `nn.Unfold`)

Worth noting, however, is that the view-based version of `nn.Unfold` has higher peak memory usage than the copy-based version in some configurations. Further testing revealed that, while the view-based version always has lower memory during the forward pass, the backward pass of ‘Unfold copy’ is sometimes able to reuse allocations from the forward pass where ‘Unfold view’ cannot, thereby using less total memory. More careful analysis and optimisation in this area may yield a version of ‘Unfold view’ that has memory usage no worse than ‘Unfold copy’, but it was not considered relevant seeing as the custom CUDA operators were clearly superior in this case.

Performance of tropical-max convolutions (RTX-5090)



Average latency



Peak memory usage

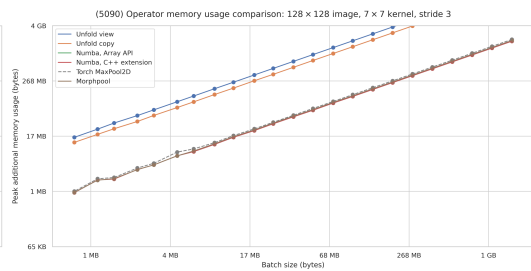
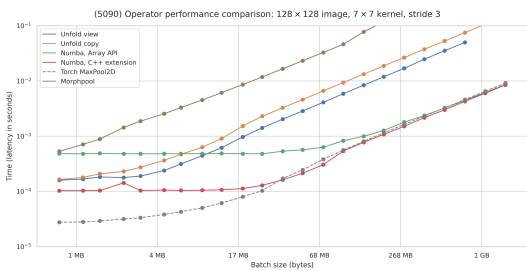
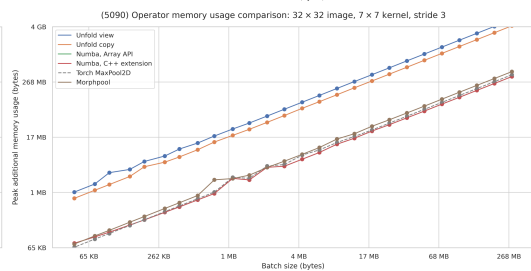
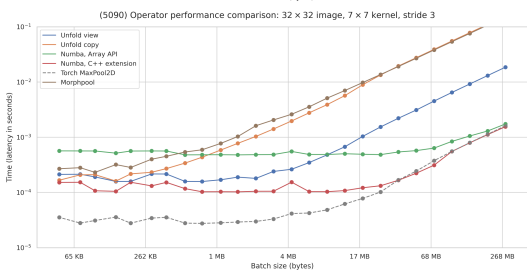
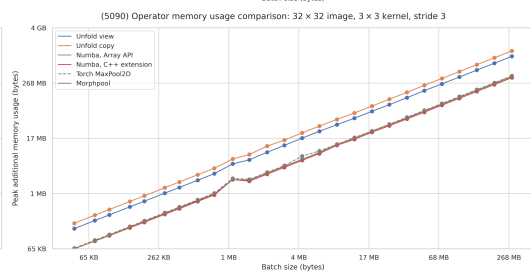
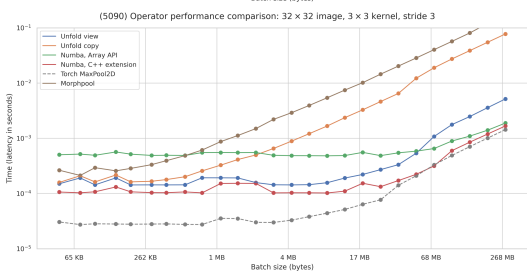
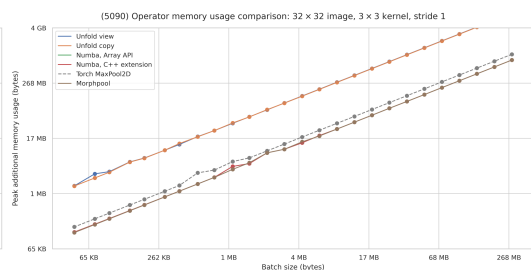


Figure 4.1: Performance of various implementations of a T_+ convolution on four configurations, with timings obtained using an NVIDIA RTX 5090. Note that ‘Peak memory usage’ includes the allocation for the result: the actual auxiliary memory used by the custom CUDA operators is a small constant.

Chapter 5

Conclusions

As part of this extension to the main report, we have seen the following:

- Using `nn.Unfold` or a view-based version thereof, built-in broadcasting operators from PyTorch can be used to implement semifield convolutions.
- With selection-based or invertible semifields, backpropagation can be done in an efficient and numerically stable manner.
- Using Just-In-Time compilation, Numba-CUDA can create specialised CUDA kernels that perform convolutions in a wide range of semifields.
- With careful tuning, convolutions in selection-based semifields can have a peak throughput similar to that of a simple max-pooling.

5.1 Discussion

Using the three types of semifields discussed in Chapter 2, most semifield convolutions can be represented within the `pytorch-nd-semiconv` package written for this report. However, the constructions implemented are not universal, and some semifields (e.g. T_+ but with $\oplus$ being the average of top-K instead of top-1) are not representable within the two variants of CUDA operators. Furthermore, even those invertible semifields representable using `SubtractSemifield` still suffer from poorly optimised performance.

If we return to the perspective of the main report, however, the performance of the selection-based semifields is quite good. Overhead remains a problem for small batch sizes, but the throughput of the custom CUDA operators from `SelectSemifield` matches that of a simple max pooling. The end result (including some optimisations in model training) is that the experiments from previous works can be replicated using almost two orders of magnitude less time, and larger experiments can be run within the same timeframe.

It is also worth noting that the flexibility of JIT compilation also allowed for generating code dynamically, such that the resulting convolutional operators work in arbitrary dimensionality. Future work that might wish to examine e.g. 3D volumetric data using dilations can therefore also benefit from this work. The cost of utilising such flexibility is, however, that future work wishing to modify the code for the package may find it challenging to understand the compilation process and make appropriate changes.

Finally, it should be mentioned that while it was considered a fairly obvious choice to restrict the semifields considered to those where the derivative can be computed without backpropagating through the reduction, it may be the case that such backpropagation is less expensive or unstable than assumed.

5.2 Contributions

To summarise the contributions of this report, we have seen:

- A description of how to reformulate two kinds of semifields in a manner which lends itself better to backpropagation.
- A short introduction of some of the background for CUDA operators.
- The package `pytorch-nd-semiconv`, capable of dynamically compiling N-dimensional convolutional operators for many semifields, where prior work was limited to fixed semifields and dimensionality.
- A performance analysis of the above package for the T_+ semifield.

5.3 Further research

While the main report has suggestions on possible directions for research into the applications of semifields for CNNs, there are also some expansions possible to the code written for this extension report which may be worth pursuing:

- Quantifying the effects of performing a full backpropagation through the reduction of a convolution (see Fig. 2.2, left). The hypothesis is that such an approach would negatively impact performance and accuracy, but the extent to which this is the case is unknown. Creating a new, generic implementation for CUDA kernels and measuring its performance may lead to new insights.
- Improving the performance of `SubtractSemifield` by using a different approach for the backward pass and possibly using shared memory or other CUDA features to accelerate computation.
- Identifying other classes of semifields which can be efficiently translated to CUDA operators (other than selection-based and invertible semifields).
- Investigating whether CUDA features improve performance in contexts other than the dilation-based poolings for small images used for the main report; larger images (or 3D data) may have different performance tradeoffs, where using e.g. shared memory may prove to be valuable.

5.4 Reproducibility

All code is available in a [GitHub repository](#), the same as the main report. The packages [pytorch-nd-semiconv](#) and [pytorch-numba-extension-jit](#) are also both listed on the Python Package Index (PyPI).

Code for generating the graphs is also available, including the SVG files for all illustrations in the report and a copy of the report's $\text{\LaTeX}$ code.

Bibliography

- [1] R. Gonzalez and R. Woods, *Digital Image Processing Global Edition*. Pearson Deutschland, 2017. [Online]. Available: <https://elibrary.pearson.de/book/99.150005/9781292223070>
- [2] H. J. Heijmans, *Morphological image operators*. Philadelphia, Society for Industrial and Applied Mathematics., 1996, vol. 38, no. 1.
- [3] J. A. Beachy, *Abstract Algebra (Third Edition)*, 2006.
- [4] P. Maragos, “Tropical Geometry, Mathematical Morphology and Weighted Lattices,” in *Mathematical Morphology and Its Applications to Signal and Image Processing*, B. Burgeth, A. Kleefeld, B. Naegel, N. Passat, and B. Perret, Eds. Cham: Springer International Publishing, 2019, pp. 3–15.
- [5] P. Adema. Anisotropic quadratic forms for pooling in deep learning. [Online]. Available: <https://github.com/p-adema/quadratic-conv/blob/main/report/report.pdf>
- [6] R. Groenendijk, L. Dorst, and T. Gevers, “Morphpool: efficient non-linear pooling & unpooling in CNNs,” *arXiv preprint arXiv:2211.14037*, 2022.
- [7] A. Paszke, M. J. Johnson, R. Frostig, and D. Maclaurin, “Parallelism-preserving automatic differentiation for second-order array languages,” in *Proceedings of the 9th ACM SIGPLAN International Workshop on Functional High-Performance and Numerical Computing*, 2021, pp. 13–23.
- [8] NVIDIA. CUDA C++ programming guide. [Online]. Available: <https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html>
- [9] NVIDIA blackwell tuning guide. [Online]. Available: <https://docs.nvidia.com/cuda/blackwell-tuning-guide/index.html>
- [10] PTX ISA 8.8 documentation. [Online]. Available: <https://docs.nvidia.com/cuda/parallel-thread-execution/index.html>
- [11] NVIDIA/numba-cuda: The CUDA target for numba. [Online]. Available: <https://github.com/NVIDIA/numba-cuda>

Appendix

A Performance results on other GPUs

Besides the NVIDIA RTX 5090 GPU used for development and primary results, three other GPUs were used to generate additional results. The following table shows an overview (all NVIDIA GPUs):

Name	Source	Release date	Results
RTX 5090	Author	Jan 2025	Fig. 4.1
H100	SURF Snellius	Mar 2023	Fig. 5.1
A100	SURF Snellius	Jun 2021	Fig. 5.2
Tesla T4	Google Colab	Sep 2018	Fig. 5.3

Table 5.1: GPUs used for obtaining results

Examining the results on the following pages, we can see that they are similar to Fig. 4.1, with older GPUs being progressively slower. Memory usage is the same between GPUs, with the only real difference being that the RTX 5090 has faster but less stable performance at low batch sizes. This difference may be due to architectural changes, or it may be due to the RTX 5090 being a consumer card while the other three are datacenter cards (hence optimised for stability).

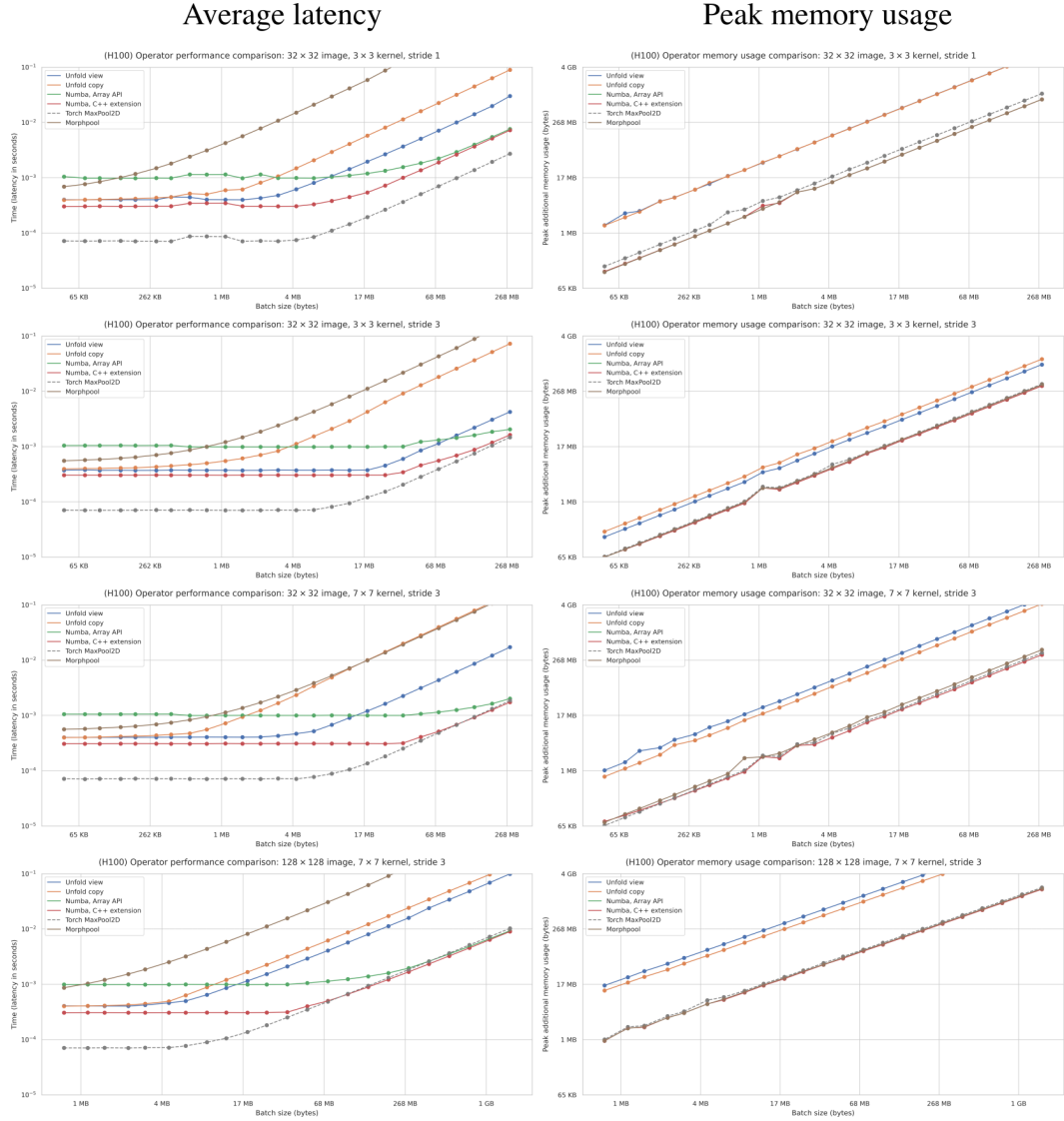


Figure 5.1: Performance of various implementations of a T_+ convolution on four configurations, with timings obtained using an NVIDIA H100. Note that 'Peak memory usage' includes the allocation for the result: the actual auxiliary memory used by the custom CUDA operators is a small constant.

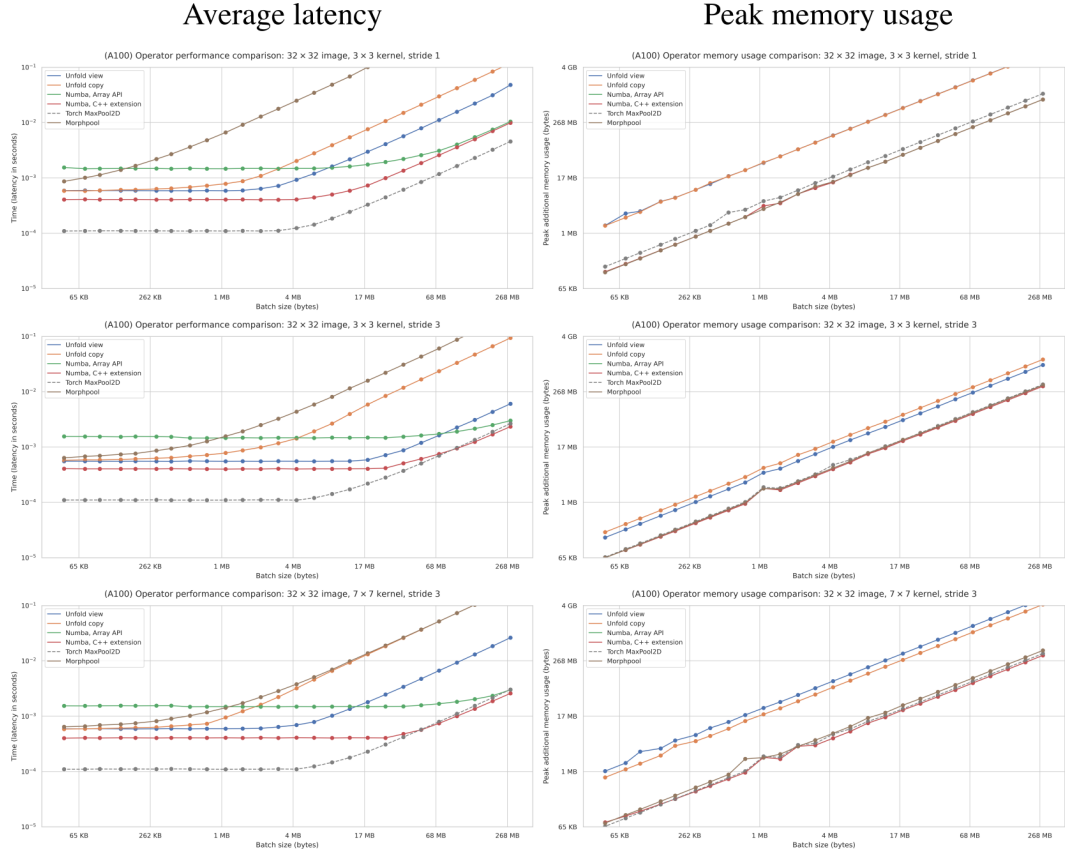


Figure 5.2: Performance of various implementations of a T_+ convolution on four configurations, with timings obtained using an NVIDIA A100.

Note that 'Peak memory usage' includes the allocation for the result: the actual auxiliary memory used by the custom CUDA operators is a small constant. Note also that the last set (128×128 images) is missing: due to an unknown error, it was not possible to run these experiments on time using the A100.

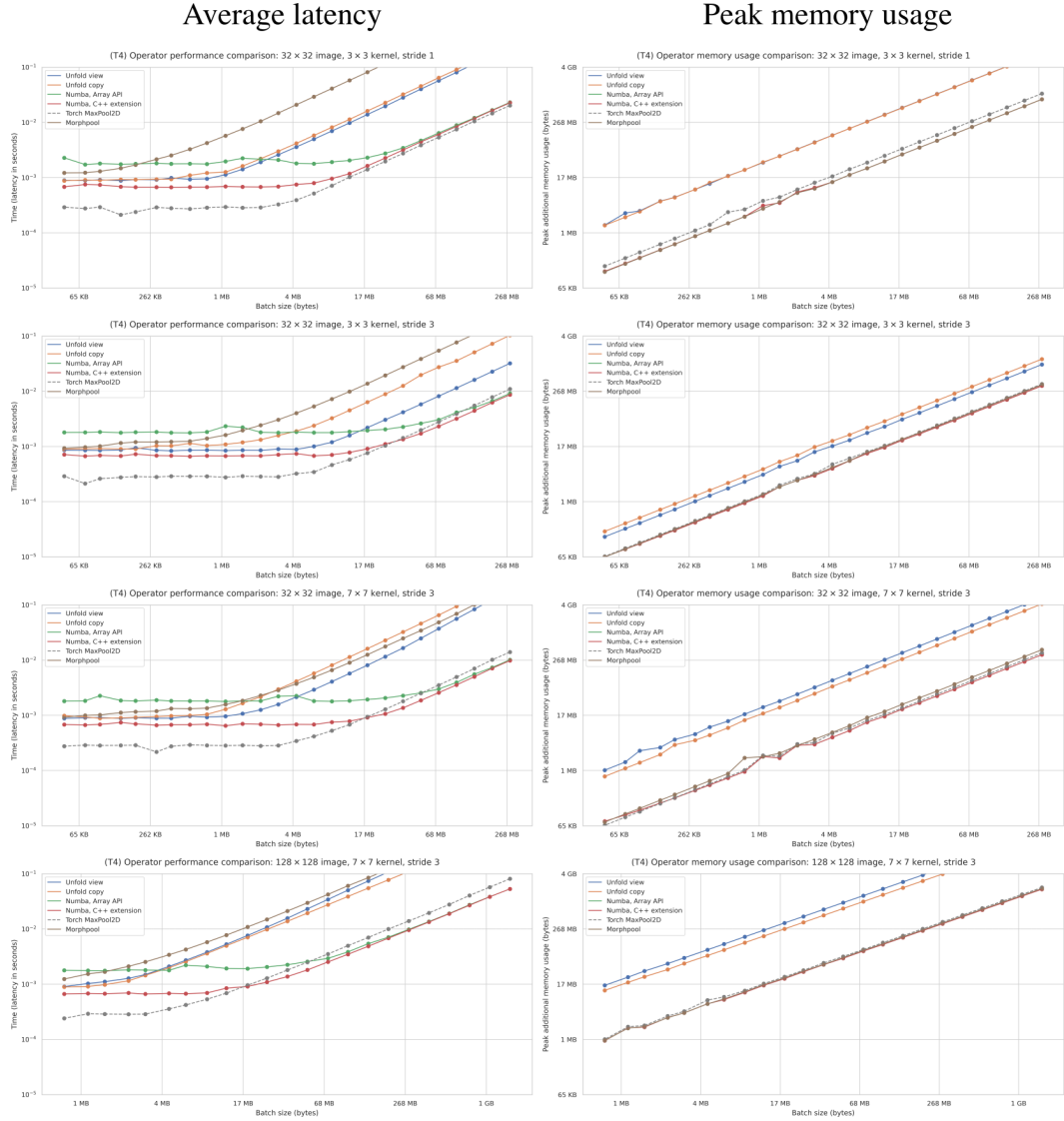


Figure 5.3: Performance of various implementations of a T_+ convolution on four configurations, with timings obtained using an NVIDIA Tesla T4. Note that 'Peak memory usage' includes the allocation for the result: the actual auxiliary memory used by the custom CUDA operators is a small constant.